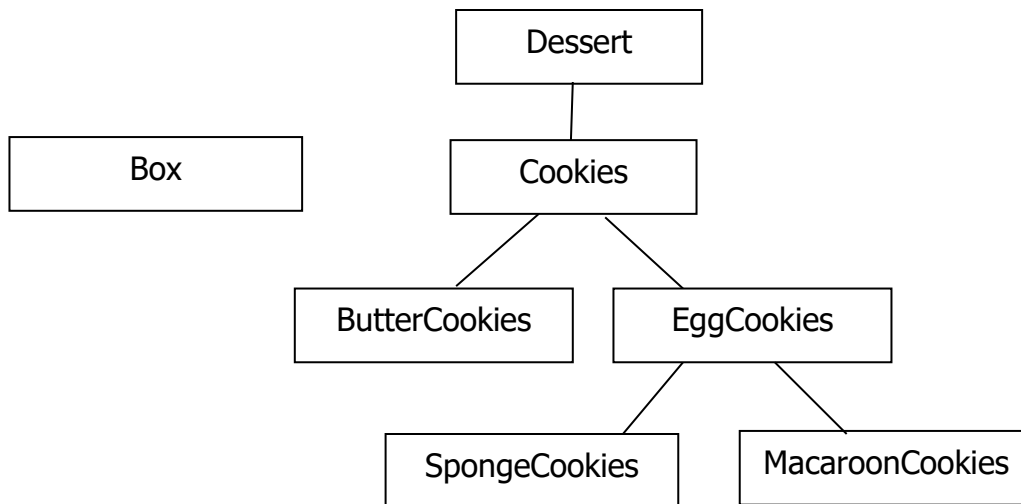


แบบฝึกหัดที่ 54

จงสร้างคลาส Dessert, Cookies, ButterCookies, EggCookies, SpongeCookies, MacaroonCookies โดยมีการรับทอดตามรูปทั้ง 6 คลาส และกำหนดให้การรับทอดทั้งหมดเป็นการรับทอดแบบ public และสร้างคลาส Box อีก 1 คลาส ทุกคลาสให้เก็บข้อมูลทั้งหมดไว้ในส่วนของ private และให้กำหนด Dessert, Cookies, และ EggCookies เป็น Abstract class ทั้ง 3 คลาส โดยให้สร้าง เพียวเวอร์ชวลฟังก์ชันที่ชื่อว่า void show()



class Dessert

ประกอบด้วยข้อมูล int calories; ปริมาณแคลอรี/ชิ้น

class Cookies

ประกอบด้วยข้อมูล int numberOfPieces; จำนวนชิ้นของคุกกี้ทั้งหมดที่ทำ
int num; จำนวนท็อปปิ้งที่ใส่ในคุกกี้แต่ละชิ้น
string *topping; อาร์เรย์เก็บท็อปปิ้ง
ขนาดของอาร์เรย์เท่ากับ num
เช่น num=2,
topping = ["Almond", "Coconut"]

class ButterCookies

ประกอบด้วยข้อมูล int type; 1.ใช้น้ำมัน 2.ใช้น้ำตาล 3.ใช้มาการีน

class EggCookies

ประกอบด้วยข้อมูล static int numEggCookies; ใช้นับจำนวนอ็อบเจกต์ของคลาส
EggCookies ทั้งหมด

```
class SpongeCookies
```

```
ประกอบด้วยข้อมูล    string name;        ชื่อของ sponge cookies
```

```
class MacaroonCookies
```

```
ประกอบด้วยข้อมูล    bool isRound;        true: เป็นก้อนกลม    false: เป็นรูปอื่น
```

```
class Box    (เป็นกล่องใช้เก็บคุกกี้ได้หลายแบบ)
```

```
ประกอบด้วยข้อมูล    int num;        ขนาดของอาร์เรย์ cookies
                    Cookies **cookies;    อาร์เรย์ของพอยเตอร์ที่ชี้ไปยัง Cookies
                                           (อาร์เรย์มีขนาดเท่ากับ num)
```

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้
(ในกรณีที่ต้องจองพื้นที่ ให้คืน.จองพื้นที่และคืนพื้นที่ให้เรียบร้อยด้วย)

ทุกคลาส จงสร้าง

- default constructor ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัว และมีการจองพื้นที่ให้ถูกต้องด้วย โดยมีการกำหนดค่าเริ่มต้นดังนี้
 - o Dessert : ปริมาณแครอว์ 100
 - o Cookies : มีคุกกี้ทั้งหมด 20 ชิ้น แต่ละชิ้นมีปริมาณแครอว์ 100 และไม่ใส่ topping
 - o ButterCookies : มีคุกกี้ทั้งหมด 20 ชิ้น แต่ละชิ้นมีปริมาณแครอว์ 100 ไม่ใส่ topping และใช้เนยเค็ม
 - o EggCookies : มีคุกกี้ทั้งหมด 20 ชิ้น แต่ละชิ้นมีปริมาณแครอว์ 100 ไม่ใส่ topping และนับจำนวนอีอบเจกต์เพิ่มทีละ 1 โดยอัตโนมัติ
 - o SpongeCookies : มีคุกกี้ทั้งหมด 20 ชิ้น แต่ละชิ้นมีปริมาณแครอว์ 100 ไม่ใส่ topping และมีชื่อว่า Italian sponge cookies
 - o MacaroonCookies : มีคุกกี้ทั้งหมด 20 ชิ้น แต่ละชิ้นมีปริมาณแครอว์ 100 ไม่ใส่ topping และเป็นก้อนกลม
 - o Box : ยังไม่มีคุกกี้อยู่ในกล่อง num=0 และ cookies=0
- constructor ในการกำหนดค่าให้กับแอทริบิวต์ทุกตัว (รวมทั้งbase class) โดยรับข้อมูลจากพารามิเตอร์
 - o กรณี topping ให้จองพื้นที่ขนาดเท่ากับ num โดยยังไม่ต้องกำหนดค่าในอาร์เรย์
 - o กรณี cookies ให้จองพื้นที่ขนาดเท่ากับ num โดยยังไม่ต้องกำหนดค่าในอาร์เรย์
 - o กรณี numEggCookies ให้นับจำนวนอีอบเจกต์เพิ่มทีละ 1 โดยอัตโนมัติ
- destructor ในการพิมพ์ค่าเฉพาะในคลาสของตนเองออกทางจอภาพ และคืนพื้นที่
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอทริบิวต์ของแต่ละคลาส เช่น setCalories
 - o กรณีแอทริบิวต์ที่เป็น static ไม่ต้องมีการกำหนดค่า
 - o กรณีที่เป็นอาร์เรย์ให้กำหนดค่าในอาร์เรย์ทีละ 1 ค่า โดยระบุตำแหน่งที่เก็บในอาร์เรย์ เช่น setTopping(int, string) เป็นต้น
 - o กรณี setNum ของคลาส Cookies ให้กำหนดค่า num รวมทั้งคืนพื้นที่เดิมและจองพื้นที่ใหม่ให้กับ topping ด้วย
 - o กรณี setNum ของคลาส Box ให้กำหนดค่า num รวมทั้งคืนพื้นที่เดิมและจองพื้นที่ใหม่ให้กับ cookies ด้วย

- ฟังก์ชัน `get` ในการรีเทิร์นแอททริบิวต์แต่ละค่า เช่น `getCalories`
 - o สำหรับ `topping` และ `cookies` ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ทีละ 1 ค่าโดยระบุตำแหน่งที่เก็บในอาร์เรย์
 - o สำหรับคลาส `EggCookies` ให้สร้างฟังก์ชัน `get` เป็น static function

คลาส `ButterCookies`, คลาส `SpongeCookies`, คลาส `MacaroonCookies`

- ให้ override ฟังก์ชัน `void show()` โดยให้แสดงข้อมูลทุกอย่างของคลาสตนเองและของ superclass ออกทางจอภาพ แต่ไม่ต้องแสดงค่า `numEggCookies`

คลาส `ButterCookies`

- จงสร้าง operator `++` แบบ prefix ในการเพิ่มค่า `calories` ทีละ 5 (ให้ทำเป็นสมาชิกของคลาส)
- สร้าง operator `<<` ในการพิมพ์ค่าทุกค่าของคลาสตนเองและของ superclass ออกทางจอภาพ

คลาส `SpongeCookies`

- จงสร้าง operator `=` ในการกำหนดค่าและจองพื้นที่อย่างถูกต้อง

คลาส `MacaroonCookies`

- จงสร้างฟังก์ชัน ในการกำหนดค่าต่างๆ โดยใช้หลักการ function overloading
- จงสร้าง operator `!` ในการเปลี่ยนค่า `isRound` เป็นค่าตรงข้ามทุกครั้งที่ใช้ (ให้ทำเป็น friend function)

คลาส `Box`

- สร้างฟังก์ชัน `int amount()` ในการรีเทิร์นค่าผลรวมของจำนวนชิ้นของคุกกี้ทั้งหมดในอาร์เรย์ `cookies`
- สร้างฟังก์ชัน `void showBox()` ในการแสดงค่าทุกค่าของอ็อบเจกต์ที่เก็บใน `cookies` โดยให้ไปเรียกใช้ฟังก์ชัน `show()` ของสมาชิกแต่ละตัวในอาร์เรย์ โดยใช้หลักการ dynamic binding
- จงสร้าง `Cookies* operator[](int index)` ในการรีเทิร์นค่าที่เก็บในอาร์เรย์ `cookies` ตำแหน่งที่ `index`

ใน main

- จงสร้างอ็อบเจกต์ `butter`, `sponge`, `macaroon` ให้เป็น instance ของ คลาส `ButterCookies`, `SpongeCookies` และ `MacaroonCookies` ตามลำดับ โดยให้สร้างโดยใช้ constructor ที่รับพารามิเตอร์ในการกำหนดค่า โดยให้กำหนดค่าดังนี้
 - o `butter` : มีทั้งหมด 40 ชิ้น แต่ละชิ้นมีปริมาณแครอว์รี 105 ใช้น้ำมัน และใส่ topping 5 อย่าง
 - o `sponge` : มีชื่อว่า `Aruba sponge cookies` มีทั้งหมด 30 ชิ้น แต่ละชิ้นมีปริมาณแครอว์รี 110 และไม่มีใส่ topping
 - o `macaroon` : เป็นก้อนกลม มีทั้งหมด 25 ชิ้น แต่ละชิ้นมีปริมาณแครอว์รี 115 และใส่ topping 2 อย่าง
- จงกำหนดค่า `topping` ให้กับ `butter` ดังนี้ `Raisin`, `Chocolate chip`, `Almond`, `Oat` และ `Cashew nut`

- จงกำหนดค่า topping ให้กับ macaroon ดังนี้ Almond และ Coconut
- จงสร้าง box เป็นอ็อบเจกต์ของคลาส Box โดยกำหนดให้มีคุกกี้ 3 ชนิด คือ butter, sponge และ macaroon ที่สร้างขึ้นก่อนหน้านี้
- จงแสดงค่าทั้งหมดใน Box โดยการเรียกใช้ฟังก์ชัน showBox
- จงสร้าง butter2 ให้เป็น instance ของคลาส ButterCookies โดยให้เรียกใช้ default constructor จากนั้นให้กำหนดค่าดังนี้ มีทั้งหมด 15 ชิ้น แต่ละชิ้นมีปริมาณแคลอรี่ 130 ใช้เนยเค็ม และใส่ topping 1 อย่าง คือ Chocolate chip
- จงเรียกใช้ operator++ เพื่อเพิ่มค่า calories ของ butter2 หลังจากนั้นให้แสดงค่าของ butter โดยใช้โอเปอเรเตอร์ <<
- จงสร้าง sponge2 ให้เป็น instance ของคลาส SpongeCookies โดยให้เรียกใช้ default constructor จากนั้นให้กำหนดค่าดังนี้ มีทั้งหมด 40 ชิ้น แต่ละชิ้นมีปริมาณแคลอรี่ 120 และใส่ topping 3 อย่าง คือ Chocolate chip, Almond, Coconut
- จงเปลี่ยนค่าของ sponge ให้เท่ากับค่าของ sponge2 จากนั้นให้แสดงค่าใหม่ของ sponge ออกทางจอภาพ
- จงสลับค่า isRound ของ macaroon โดยเรียกใช้โอเปอเรเตอร์ !
- จงเปลี่ยนค่า num ของ box จาก 3 ให้เป็น 5 และกำหนดค่าให้กับ cookies ใหม่ตามลำดับดังนี้ butter, butter2, sponge, sponge2, macaroon
- จงแสดงค่าทั้งหมดของ box ออกทางจอภาพ
- ในโปรแกรมตอนนี้มีอ็อบเจกต์ที่เป็น EggCookies ทั้งหมดก็อ็อบเจกต์
- จงหาผลรวมของ calories ของคุกกี้ทุกชิ้นที่เก็บอยู่ใน box[1]

(หมายเหตุ: ทุกๆ object จะต้องถูกสร้างและถูกทำลายได้อย่างถูกต้อง และน.ศ.สามารถสร้าง attributes และ function เพิ่มเติมได้ตามความเหมาะสม)

เฉลย

```
#include<iostream>
using namespace std;

class Dessert {
    int calories;
public:
    Dessert() {
        setCalories(100);
    }
    Dessert(int c) {
        setCalories(c);
    }
    ~Dessert() {
        cout<<"calories: "<<calories<<endl;
    }
    void setCalories(int c) {
        calories=c;
    }
    int getCalories() {
        return calories;
    }
    virtual void show() = 0;
};

class Cookies: public Dessert {
    int numberOfPeices;
    int num;
    string *topping;
public:
    Cookies() {
        setNumberOfPeices(20);
        num=0;
        topping = 0;
    }
    Cookies(int c, int p, int n): Dessert(c) {
        setNumberOfPeices(p);
        num=n;
        topping = new string[num];
    }
    ~Cookies() {
        showCookies();
        delete[] topping;
    }
}
```

```

void showCookies() {
    cout<<"number of peices: "<<numberOfPeices<<endl;
    cout<<"number of topping: "<<num<<endl;
    if (num!=0) {
        cout<<"Topping: "<<endl;
        for (int i=0; i<num; i++) {
            cout<<" "<<topping[i]<<endl;
        }
    }
}

void setNumberOfPeices(int n) {
    numberOfPeices = n;
}

void setNum(int n) {
    if (num == n) return;
    if (topping != 0) {
        delete[] topping;
    }
    num = n;
    if (num==0)
        topping=0;
    else
        topping = new string[num];
}

void setTopping(int i, string n) {
    topping[i] = n;
}

int getNumberOfPeices() {
    return numberOfPeices;
}

int getNum() {
    return num;
}

string getTopping(int index) {
    return topping[index];
}

virtual void show() = 0;
};

```

```

class ButterCookies: public Cookies {
    int type;
public:
    ButterCookies() {
        setType(1);
    }
    ButterCookies(int c, int p, int n, int t): Cookies(c,p,n) {
        setType(t);
    }
    ~ButterCookies() {
        cout<<"butter cookies die...."<<endl;
        cout<<"type: "<<type<<endl;
    }
    void setType(int t) {
        type = t;
    }
    int getType() {
        return type;
    }
    void show() {
        cout<<"calories: "<<getCalories()<<endl;
        cout<<"type: "<<type<<endl;
        showCookies();
    }
    void operator++() {
        setCalories(getCalories()+5);
    }
};

ostream& operator<<(ostream &o, ButterCookies &b) {
    b.show();
    return o;
}

```

```

class EggCookies: public Cookies {
    static int numEggCookies;
public:
    EggCookies() {
        numEggCookies++;
    }
    EggCookies(int c, int p, int n): Cookies(c,p,n) {
        numEggCookies++;
    }
}

```

```

~EggCookies() {
    numEggCookies--; //โจทย์ไม่ได้สั่ง จะทำหรือไม่ทำก็ไม่ว่าอะไร
    cout<<"number of Egg Cookies: "<<numEggCookies<<endl;
}
static int getNumEggCookies() {
    return numEggCookies;
}
virtual void show() = 0;
};
int EggCookies:: numEggCookies;

class SpongeCookies: public EggCookies {
    string name;
public:
    SpongeCookies() {
        name = "Italian sponge cookies";
    }
    SpongeCookies(int c, int p, int n, string m): EggCookies(c,p,n) {
        name = m;
    }
    ~SpongeCookies() {
        cout<<"sponge cookies die..."<<endl;
        cout<<"name: "<<name<<endl;
    }
    void setName(string n) {
        name = n;
    }
    string getName() {
        return name;
    }
    void show() {
        cout<<"calories: "<<getCalories()<<endl;
        cout<<"name: "<<name<<endl;
        showCookies();
    }
    SpongeCookies& operator=(SpongeCookies &c) {
        if (this==&c) return *this;
        name=c.getName();
        setCalories(c.getCalories());
        setNumberOfPeices(c.getNumberOfPeices());
        setNum(c.getNum());
    }

```

```

        for (int i=0; i<getNum(); i++) {
            setTopping(i, c.getTopping(i));
        }
        return *this;
    }
};

class MacaroonCookies: public EggCookies {
    bool isRound;
public:
    MacaroonCookies() {
        isRound = true;
    }
    MacaroonCookies(int c, int p, int n, bool b): EggCookies(c,p,n) {
        isRound = b;
    }
    ~MacaroonCookies() {
        cout<<"macaroon cookies die..."<<endl;
        showMacaroonCookies();
    }
    void setIsRound(bool b) {
        isRound = b;
    }
    bool getIsRound() {
        return isRound;
    }
    void showMacaroonCookies() {
        cout<<"round?"<<(isRound?" yes": " no")<<endl;
    }
    void show() {
        cout<<"calories: "<<getCalories()<<endl;
        showMacaroonCookies();
        showCookies();
    }
    void setMacaroon(int c) {
        setCalories(c);
    }
    void setMacaroon(int p, bool b) {
        setNumberOfPeices(p);
        setIsRound(b);
    }
    friend void operator!(MacaroonCookies &m) {
        m.setIsRound(!m.getIsRound());
    }
};

```

```

class Box {
    int num;
    Cookies **cookies;
public:
    Box() {
        num=0;
        cookies=0;
    }
    Box(int n) {
        num = n;
        cookies = new Cookies*[num];
    }
    ~Box() {
        cout<<"Box num: "<<num<<endl;
        delete[] cookies;
    }
    int getNum() {
        return num;
    }
    Cookies* getCookies(int index) {
        return cookies[index];
    }
    void setNum(int n) {
        if (num == n) return;
        if (cookies != 0) {
            delete[] cookies;
        }
        num = n;
        if (num==0)
            cookies=0;
        else
            cookies = new Cookies*[num];
    }
    void setCookies(int i, Cookies* c) {
        cookies[i] = c;
    }
    int amount() {
        int total = 0;
        for (int i=0; i<num; i++) {
            total+=cookies[i]->getNumberOfPeices();
        }
        return total;
    }
}

```

```

void showBox() {
    for (int i=0; i<num; i++) {
        cookies[i]->show();
        cout<<endl;
    }
}
Cookies* operator[](int index) {
    return cookies[index];
}
};

int main() {
    ButterCookies butter(105, 40, 5, 2);
    SpongeCookies sponge(110, 30, 0, "Aruba sponge cookies");
    MacaroonCookies macaroon(115, 25, 2, true);
    butter.setTopping(0, "Raisin");
    butter.setTopping(1, "Chocolate chip");
    butter.setTopping(2, "Almond");
    butter.setTopping(3, "Oat");
    butter.setTopping(4, "Cashew nut");
    macaroon.setTopping(0, "Almond");
    macaroon.setTopping(1, "Coconut");

    Box box(3);
    box.setCookies(0, &butter);
    box.setCookies(1, &sponge);
    box.setCookies(2, &macaroon);

    ButterCookies butter2;
    butter2.setNumberOfPeices(15);
    butter2.setCalories(130);
    butter2.setType(1);
    butter2.setNum(1);
    butter2.setTopping(0, "Chocolate chip");
    ++butter2;
    cout<<butter2;

    SpongeCookies sponge2;
    sponge2.setNumberOfPeices(40);
    sponge2.setCalories(120);
    sponge2.setNum(3);
    sponge2.setTopping(0, "Chocolate chip");
    sponge2.setTopping(1, "Almond");

```

```

    sponge2.setTopping(2, "Coconut");
    cout<<endl;
    sponge2.show();

    sponge = sponge2;
    sponge.show();

    !macaroon;
    cout<<endl;
    macaroon.show();

    cout<<endl;
    cout<<box.amount()<<endl;

    box.setNum(5);
    box.setCookies(0, &butter);
    box.setCookies(1, &butter2);
    box.setCookies(2, &sponge);
    box.setCookies(3, &sponge2);
    box.setCookies(4, &macaroon);

    cout<<"show box..."<<endl;
    box.showBox();

    cout<<"number of egg cookies "<<EggCookies::getNumEggCookies()<<endl;

    cout<<endl;
    int allCalories = 0;
    allCalories = box[1]->getCalories() * box[1]->getNumberOfPeices();
    cout<<"all calories: "<<allCalories<<endl;
    cout<<"-----"<<endl;
}

```